

TaskFlow — Backend Developer Technical Assignment

Problem Brief

Build the backend for **TaskFlow**, a lightweight project management system where users belong to organizations, create projects, manage tasks, assign work, and receive background notifications.

The solution should demonstrate **secure multi-tenant access, clean architecture, database design, background jobs, testing, and production readiness.**

Technology Requirements

Category	Requirement
Language	Node.js - Express / Fastify
Optional	Python - FastAPI / Django may be used only for supporting services
Database	PostgreSQL only
Job Queue	Redis + BullMQ
Optional Worker	Celery + Redis
ORM	Prisma, TypeORM, Drizzle, SQLAlchemy, etc.
Containers	Docker Compose
Required Services	API, Worker, PostgreSQL, Redis

Not allowed: SQLite, MongoDB, Firebase, Supabase, PlanetScale, or other BaaS solutions.

Task 01 - Data Modeling & Database Design

Design and implement the PostgreSQL schema.

Required Tables

- users
- organizations
- org_members
- projects
- tasks
- task_assignments
- comments

Requirements

- Proper foreign keys with documented `CASCADE / RESTRICT` decisions
- Project must reference organizations.
- Tasks must reference projects.
- task_assignments must reference both “task” and “users”.
- Comments must reference both “tasks” and “users” (author)
- PostgreSQL enums:
 - Status: `todo, in_progress, review, done`
 - Priority: `low, medium, high, urgent`
- Index frequently queried columns and justify indexes in code comments
- Use migration files — no manually maintained `schema.sql`
- Include migration up/down support where applicable
- Seed data:
 - 2 organizations
 - 5 users
 - Multiple projects
 - 10+ tasks
 - Tasks distributed across projects
 - Different statuses/priorities
 - Assignments and sample comments

Bonus

- ★ Soft delete using `deleted_at` on projects/tasks
 - ★ PostgreSQL full-text search on task title + description
-

Task 02 - Authentication & Authorization

Implement secure authentication with organization-level RBAC.

Requirements

- Implement the following authentication endpoints:
 - POST /auth/register
 - POST /auth/login
 - POST /auth/refresh
 - POST /auth/logout
- Register and login
- bcrypt password hashing with cost factor ≥ 12
- JWT access token — **15 min TTL**
- Refresh token — **7 days**
- Store refresh tokens in DB with revocation support
- Roles:
 - `org_admin`
 - `member`
- Admins can manage members and delete projects
- JWT middleware attaches authenticated user + organization context
- All service-layer queries must be scoped by `org_id`
- Do not trust client-provided `org_id`
- Cross-tenant access must be prevented, attempts to access resource belonging to another organization must return “403 forbidden” and must not expose sensitive resource data.
- - Authentication endpoints (`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`) must be rate-limited to **10 requests/minute/IP**

Bonus

- Refresh token rotation
 - Logout all devices
-

Task 03 - REST API - Projects & Tasks

Implement clean **Route** → **Controller** → **Service** → **Data** separation.

Requirements

- Full CRUD for projects
- Full CRUD for tasks
- Every project must belong to the authenticated user's organization

- Every task must belong to a project within the authenticated user's organization
- Task filters:
 - Status
 - Priority
 - Assignee
 - Due-date range

Implement Cursor-based or offset pagination

For offset pagination, return:

```
{
  "data": [],
  "total": 0,
  "page": 1,
  "limit": 20
}
```

For cursor pagination, return

```
{
  "data": [],
  "next_cursor": null
}
```

- Zod validation for TypeScript implementations
- Consistent errors:

```
{
  "error": "Task not found",
  "code": "TASK_NOT_FOUND",
  "details": {}
}
```

- Provide endpoints to assign and unassign a user from a task.
- The assigned user must belong to the same organization as the task.
- Project dashboard with task counts grouped by status

Bonus

- Bulk task status update
 - Full-text task search
-

Task 04 - Background Jobs & Email Notifications

When a task is assigned, trigger an asynchronous email notification.

Requirements

- Use **Redis + BullMQ**
- The assignment endpoint must persist the task assignment and enqueue the email notification job before returning a successful response. Email processing must occur asynchronously in the worker and must not block the API request.
- The assignment operation must handle failures during job enqueueing without leaving the task assignment in an inconsistent state. Document the chosen consistency strategy.
- Worker processes email jobs
- Mock email sending is acceptable
- Retry failed jobs **3 times**
- Exponential backoff:
 - 1s → 2s → 4s
- After 3 retry attempts are exhausted, failed jobs must be moved to a dead-letter queue and their job status must be reported as “failed”.
- Implement:

GET /jobs/:id

Supported status:

- pending
- active
- completed
- failed
- Docker Compose must start API + Worker + PostgreSQL + Redis

The endpoint must return the job ID, current status, and relevant job metadata in a consistent JSON response.

Bonus

- Deduplicate assignments within 5 seconds
- Global email rate limit — 50 emails/minute

Task 05 - Testing & API Documentation

Testing

Implement meaningful automated tests.

Unit tests:

- Authentication logic
- Task assignment validation
- Pagination helper

Integration tests:

- Login flow
- Task CRUD
- Cross-tenant access attempt → must return **403**
- Validation/error scenarios

Test Isolation

Each test must use a clean state through:

- Transactions
- Dedicated test DB
- Or another reliable isolation strategy

API Documentation

Provide accurate:

- OpenAPI / Swagger
- Swagger UI accessible locally
- Postman or Bruno collection
- Collection must import and work without manual edits

Bonus

- ★ Coverage report
- ★ Test that task assignment creates a queue job

Evaluation Focus

- Clean architecture and naming
- Correct authentication & authorization
- Strong multi-tenant isolation
- Reliable background jobs/retries
- Normalized database and indexes
- Meaningful tests

- Working Docker setup
 - Clear README and technical decisions
-

Submission Guidelines for the Assignment

1. GitHub Repository

Public GitHub repo with clean code and [README.md](#).

2. Architecture Document

Brief system architecture, components, technologies, and data flow.

3. API Documentation

API endpoints with request/response details. Swagger/OpenAPI preferred.

4. Screen Recording & Demo

Short video demonstrating the project, key features, APIs, and important implementation decisions.

5. Setup Instructions

Steps to run the project locally, including required environment variables.

6. Security

Do not commit passwords, API keys, or sensitive credentials.

7. Submission

Share all links/documents in one place and ensure they are accessible.